

	REPORT		REF.	N/A
	TITLE	Cooling pump flow rate adjustment		
	WRITTEN BY	Giuseppe Martire @19387525		
	DATE	12/10/2025	PROJECT	
	KEYWORDS			

Documentation Objectives

Drivetrain section requested the possibility to adjust the amount of coolant flowing through the pump, with a regulation action from Controls and Systems that accounts for the following aspects:

- 1) Whenever the car is on full power, the amount of coolant flowing should be maximum.
- 2) An EVENT MODE implementation: coolant flow rate of the pump is controlled using empirically calibrated values to provide effective cooling under real competition conditions. Validation through testing is required to ensure optimal operation and the creation of a map for this mode.

Implementation overview

1) Full Power → Max Coolant Flow

A trigger condition should be set based on throttle sensor, torque sensor or power output, considering hysteresis to avoid repeated switching.

```
if throttle_position > threshold || torque_request > threshold || P_out > threshold
```

```
    pump_duty_cycle = MAX_VALUE
```

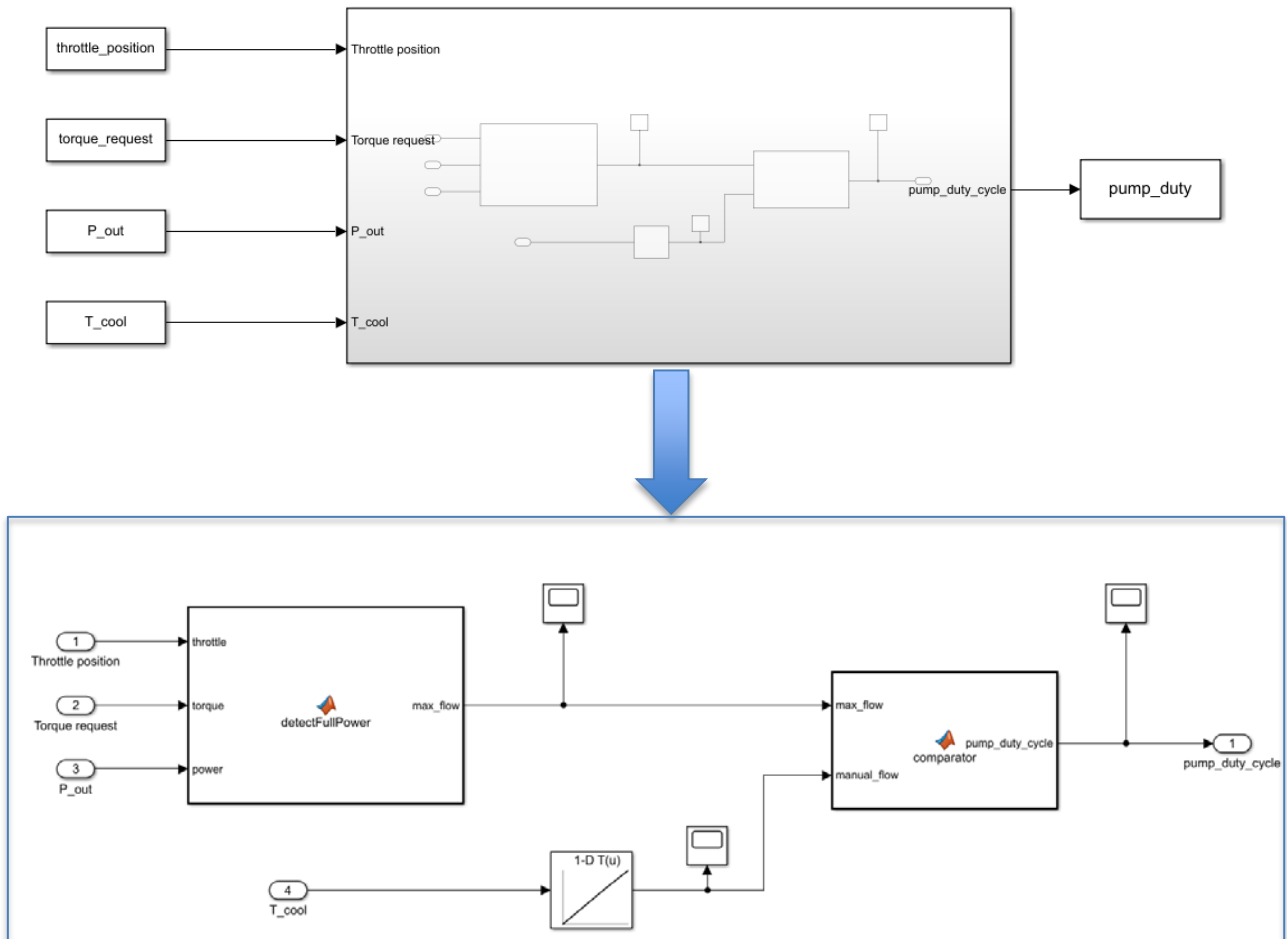
```
end
```

2) EVENT MODE implementation:

When Event Mode is desired, a predefined empirical cooling map must control the flow depending on the coolant temperature (or eventually inverter and rotor temperature, depending on which one is more critical) . It can be implemented through a **Lookup Table** using track-tested calibration data.

	REPORT		REF.	
	TITLE	Cooling pump flow rate adjustment		

Simulink model



%% INPUTS

The inputs should come from sensors as signals changing in time. Here, they are assumed to be constant values for the aim of testing the model.

They must be changed in order to test the 2 different scenarios:

1st scenario: at least one of the input parameters overcomes the limit conditions: this should trigger the maximum flow rate coming out from the first MATLAB function block.

```

Throttle_position = 0.8;      % [0-1]
Torque_request    = 350 ;     % [Nm]
P_out             = 80;       % [kW]
T_cool            = 60;       % [°C]

```

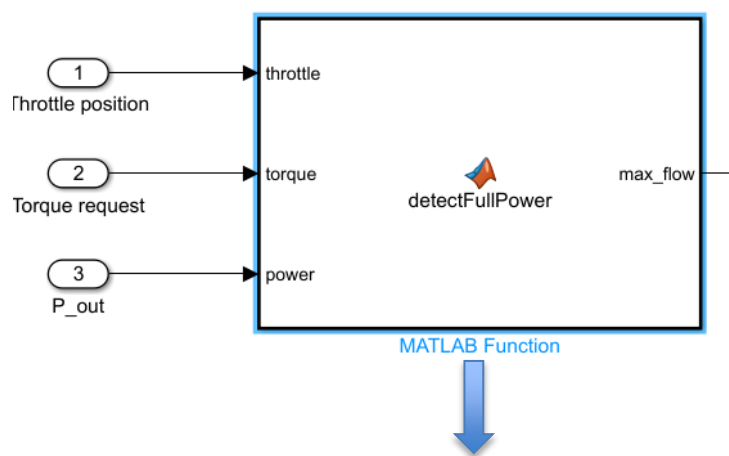
	REPORT		REF.	
	TITLE	Cooling pump flow rate adjustment		

2nd scenario: : none of the input parameters overcomes the limit conditions: this is not going to trigger the maximum flow rate coming out from the first MATLAB function block.

```

Throttle_position = 0.4;      % [0-1]
Torque_request = 200 ;       % [Nm]
P_out = 40;                  % [kW]
T_cool = 60;                 % [°C]

```



```

function max_flow = detectFullPower(throttle, torque, power)

THROTTLE_THRESHOLD = 0.8; %to be tuned for our car
TORQUE_THRESHOLD = 300;   %to be tuned
POWER_THRESHOLD = 70;     %to be tuned

if      (throttle > THROTTLE_THRESHOLD) || ...
        (torque > TORQUE_THRESHOLD) || ...
        (power > POWER_THRESHOLD)
    max_flow = 1;
else
    max_flow = 0;
end

```

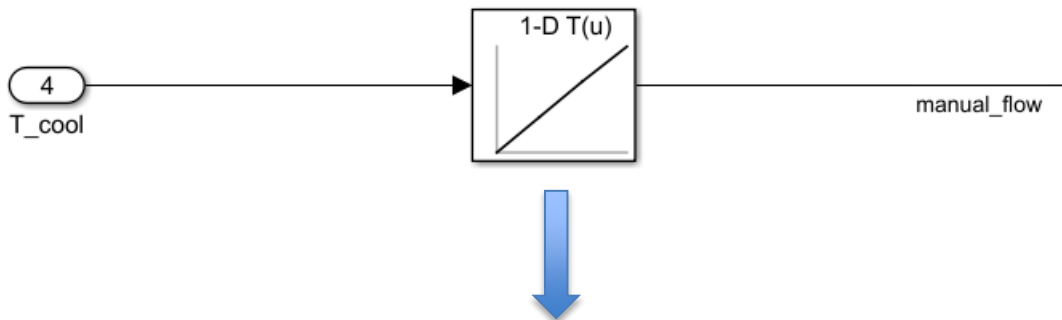
1st scenario: max_flow = 1

2nd scenario: max_flow = 0

Once this max flow signal is evaluated as input of the first MATLAB function, it is compared to the signal coming from the EVENT-MODE Lookup table.

The LUT is needed to extract a normalized flow rate value depending on the coolant temperature (which is provided as 4th input of the entire block)

	REPORT		REF.	
	TITLE	Cooling pump flow rate adjustment		



Lookup Table (n-D)

Perform n-dimensional interpolated table lookup including index searches. The table is a sampled representation of a function in N variables. Breakpoint sets relate the input values to positions in the table. The first dimension corresponds to the top (or left) input port.

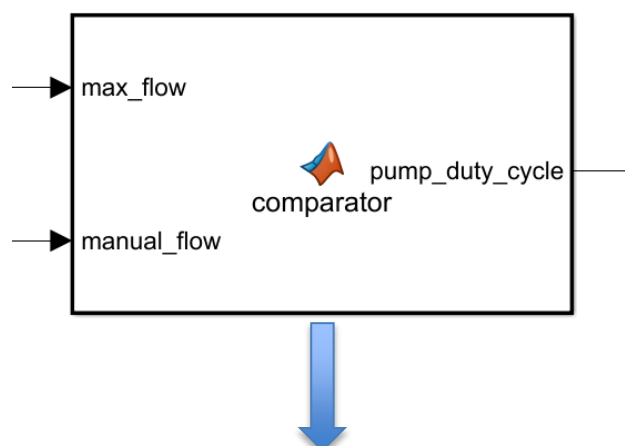
Table and Breakpoints	Algorithm	Data Types
Number of table dimensions:	1	
Data specification:	Table and breakpoints	
Table data:	event_map(:,2)	
Breakpoints specification:	Explicit values	
Breakpoints 1:	event_map(:,1)	
Edit table and breakpoints...		

% Columns: [Coolant temperature, Pump Duty]

```
event_map = [60  0.4;
             70  0.6;
             80  0.8;
             90  1.0];
```

%% EVENT MODE MAP is an empirical map, to be tuned using normalized flow rate (right column) that allows good performance at the correspondent temperature (left column)

The second MATLAB block function compares the max_flow value and the manual_flow one:



	REPORT		REF.	
	TITLE	Cooling pump flow rate adjustment		

```
function pump_duty_cycle = comparator(max_flow, manual_flow)

if max_flow > manual_flow
    pump_duty_cycle = max_flow;
else
    pump_duty_cycle = manual_flow;
end
```

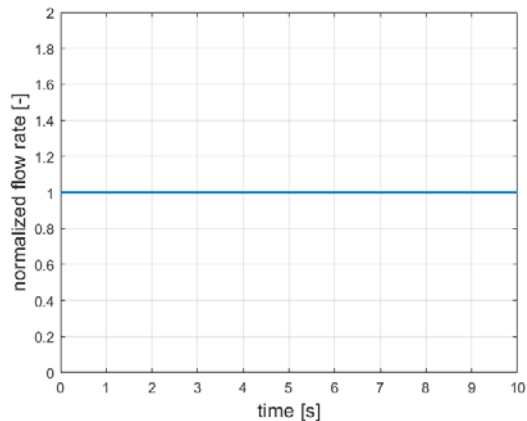
This block ensures that only when threshold parameters are overcome, $\text{max_flow} = 1$ is taken as pump_duty_cycle . Otherwise manual_flow coming out from the LUT is taken, being it always bigger than max_flow when it is equal to 0.

Simulating both scenarios, the normalized flow rate through the pump can be visualized as it follows (on MATLAB or through the last scope inside the model):

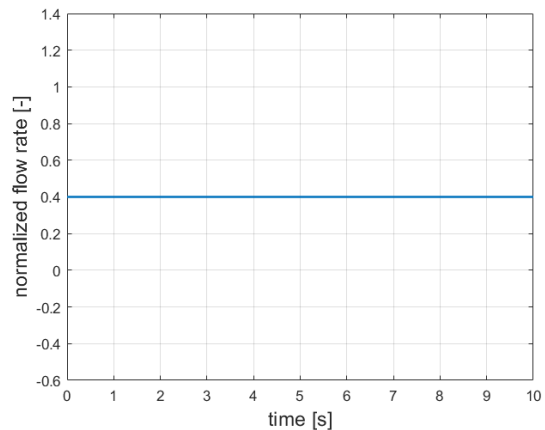
```
open("flow_adjustment.slx")
sim("flow_adjustment.slx")

plot(pump_duty.time, pump_duty.Data)
```

Testing 1st scenario:



Testing 2nd scenario:



In order to obtain significant results (rather than these constant values at the output) it is reminded that the model should be tested with real input signals coming from sensors.

	REPORT		REF.	
	TITLE	Cooling pump flow rate adjustment		

Simulink model improvement: Matlab functions are substituted using Simulink blocks

